Multi-level Inheritance: Wetland Site Management

Problem Statement

Design a C++ program that uses **multi-level inheritance** to model wetland sites. A common base class stores general wetland information, and further classes are derived step by step to add more specific details. Each level reuses the data and functions of the level above it.

Class Design

Base class: `WetlandSite`

Stores the general information shared by all wetland sites.

Data members

- `string wetlandName` : the name of the wetland
- `string location` : where the wetland is found

Member functions

- `enterSiteDetails()` : reads the wetland name and location from the user
- `displaySiteDetail()` : displays the wetland name and location

Derived classes

> Replace this section with your own derived classes from your code (`inheritance.cpp`).

Level	Class	Inherits from	Adds
1	`WetlandSite`	none (base class)	`wetlandName` , `location`
2	`[DerivedClass1]`	`WetlandSite`	`[new data and functions]`
3	`[DerivedClass2]`	`[DerivedClass1]`	`[new data and functions]`

OOP Concepts Demonstrated

- **Inheritance:** derived classes reuse the members of their parent classes.

- **Multi-level inheritance:** a class is derived from a class that is itself derived from another class.
- **Code reuse:** `enterSiteDetails()` and `displaySiteDetail()` are written once in the base class and used by all lower levels.
- **Encapsulation:** each class keeps its own data and the functions that work on it.

Program Flow

1. The program creates an object of the most derived class.
2. The user enters the site details (name and location, then the extra details from the derived classes).
3. The program displays all the details using the inherited display functions.

Diagram

Source Code

See `inheritance.cpp` in this folder.

How to Compile and Run

```
g++ inheritance.cpp -o inheritance
./inheritance
```

On Windows, run `inheritance.exe` instead of `./inheritance`.